MTOKEN · BÁO CÁO NGHIÊN CỨU · BẢN NHÁP 05/11/2026

Đo hiệu suất sử dụng token từ log cục bộ của các công cụ lập trình AI

[Tên nhóm] · [Tên thành viên] · Phiên bản 0.4

TÓM TẮT

Số token và số tiền chỉ cho biết đã tiêu bao nhiêu, không cho biết tiêu có đáng không. Báo cáo này đề xuất cách µToken đo hiệu suất sử dụng token từ log mà Codex, Claude Code và Antigravity ghi trên máy người dùng. Hiệu suất được định nghĩa là chi phí cho mỗi task thành công, luôn đi cùng tỷ lệ thành công và độ ổn định. Chúng tôi kiểm tra log thật và xác định những chỉ số tính được cho từng công cụ, đề xuất quy trình 8 bước chạy cục bộ, và một quy trình so sánh 6 bước dựa trên khoảng tin cậy và biên Pareto. LLM judge chỉ dùng để giải thích các ca bất thường, không dùng để chấm điểm hay so sánh.

Trên dữ liệu hiện có (một phiên mỗi model), **chưa thể kết luận model nào hiệu quả hơn**. Báo cáo nêu rõ cần bao nhiêu task để kết luận được: khoảng 250 task mỗi model, trong mỗi loại việc, để phát hiện chênh lệch 10 điểm phần trăm.

Từ khóa: hiệu suất token, chi phí mỗi task thành công, prompt caching, biên Pareto, khoảng tin cậy, LLM-as-a-judge

1 Giới thiệu

1.1 Vấn đề

Hai phiên cùng tốn 1 USD. Phiên thứ nhất sửa xong lỗi sau 5 lượt. Phiên thứ hai chạy một lệnh lỗi lặp đi lặp lại rồi bỏ dở. Hai con số giống nhau nhưng giá trị nhận được khác hẳn. Các công cụ theo dõi hiện nay thường chỉ đếm token hoặc cộng tiền, nên không phân biệt được hai trường hợp này.

Bên cạnh đó, chi phí một agent lập trình phụ thuộc nhiều vào cách dùng: có cache hay không, chọn model và mức suy luận nào, phiên kéo dài bao lâu. Người dùng không có cách nào biết cách dùng nào của mình là hiệu quả.

1.2 Câu hỏi nghiên cứu

1. **RQ1.** Log cục bộ của Codex, Claude Code và Antigravity ghi những gì, và đủ để tính những chỉ số nào?
2. **RQ2.** Làm sao biết một cách dùng (model, mức suy luận, công cụ, thói quen) hiệu quả và ổn định hơn cách khác?
3. **RQ3.** Vì sao tính theo cách đó? Bằng chứng nào ủng hộ từng lựa chọn?

1.3 Đóng góp

- Bảng kiểm kê trường dữ liệu trong log thật của ba công cụ, kèm hai lỗi parser dễ mắc (mục 3).
- Bộ 8 chỉ số và quy trình 8 bước chạy hoàn toàn trên máy người dùng (mục 4).
- Quy trình so sánh 6 bước, có kiểm tra cỡ mẫu và độ ổn định (mục 5).
- Kết quả đo trên log thật và câu trả lời trung thực cho câu hỏi "model nào tốt hơn" (mục 6–7).
- Mẫu kết quả hiển thị cho người dùng, với luật ra kết luận rõ ràng (mục 9).

2 Kiến thức nền

2.1 Thuật ngữ

Token Mẩu chữ model đọc và viết, cỡ một phần của từ. Giá tính theo token. Tiếng Việt thường tốn nhiều token hơn tiếng Anh cho cùng nội dung.	**Input** Mọi thứ gửi cho model ở một lần gọi: hướng dẫn hệ thống, lịch sử, kết quả lệnh, nội dung file.
Cache đọc Phần input giống hệt lần trước, được nhà cung cấp nhớ sẵn. Giá khoảng 10% input thường.	**Cache ghi** Lần đầu lưu một đoạn input vào cache. Đắt hơn input thường (×1,25 hoặc ×2 tùy thời gian giữ).

Output Chữ model viết ra. Giá mỗi token cao hơn input nhiều lần.	**Suy luận (reasoning, thinking)** Phần model nghĩ nháp trước khi trả lời. Tính tiền như output.
Lần gọi, turn, task, phiên Lần gọi: một yêu cầu tới model. Turn: một lượt hỏi tới khi agent dừng. Task: một việc người dùng muốn xong. Phiên: cả cuộc làm việc.	**Cách dùng (setup)** Tổ hợp được so sánh: công cụ, model, mức suy luận, hoặc một thói quen làm việc.

2.2 Cấu trúc chi phí

Chi phí một lần gọi gồm phần input và phần output [5]. Với prompt caching, phần input chia thành input mới, cache đọc và cache ghi, mỗi phần một hệ số giá [12]:

```
chi phí = giá_in × (input mới + h_đọc × cache đọc + h_ghi × cache ghi)
        + giá_out × output

Anthropic: h_đọc = 0,1 (Opus 5.5: 0,05); h_ghi = 1,25 (5 phút) hoặc 2 (1 giờ)
OpenAI:    h_đọc = 0,1;                  h_ghi = 1,25 (30 phút) hoặc 2,5 (12
```

Với agent, input thường chiếm phần lớn chi phí: trên τ-bench, 95,9% chi phí agent là token input [3]. Vì vậy tỷ lệ cache ảnh hưởng mạnh tới hóa đơn.

2.3 Các nguồn lãng phí đã biết

- **Input lặp không được cache.** Agent gửi lại toàn bộ lịch sử ở mỗi lần gọi [3].
- **Suy nghĩ quá mức.** Hơn 92% câu trả lời đúng đã đúng ngay ở vòng suy luận đầu; model suy luận dùng thêm khoảng 1.953% token cho câu "2+3" [4].
- **Context quá dài.** Model kém hơn khi thông tin nằm giữa context dài; tăng từ 20 lên 50 tài liệu chỉ thêm khoảng 1–1,5% độ chính xác [6].
- **Dùng model mạnh cho việc dễ.** Cascade giữ chất lượng ngang model tốt nhất với chi phí giảm 50–98% [5].
- **Lỗi và thử lại.** Mỗi lần thất bại kéo theo một lần gọi với toàn bộ context.

2.4 Nghiên cứu liên quan

Đánh giá có kiểm soát chi phí. Kapoor và cộng sự chỉ ra rằng chỉ báo độ chính xác là sai lệch, vì thử lại nhiều lần đủ làm điểm tăng; các agent có độ chính xác tương đương chênh nhau gần 100 lần về chi phí. Họ đề xuất trình bày trên biên Pareto độ chính xác – chi phí [1]. HELM cũng đánh giá nhiều chỉ số cùng lúc thay vì một con số [11].

Thống kê trong đánh giá. Miller coi bộ đề là một mẫu, đề xuất báo sai số chuẩn, so theo cặp, sai số theo cụm và tính trước cỡ mẫu [2].

Độ ổn định. τ-bench đề xuất pass^k: xác suất thành công cả k lần trên cùng task, và cho thấy chỉ số này giảm nhanh theo k [3].

LLM-as-a-judge. Giám khảo LLM mở rộng được và đồng thuận với người ở mức khá [7], nhưng có thiên vị vị trí [7][8] và tự ưu ái [9]. Mức tin cậy còn tranh cãi giữa các nghiên cứu.

Hiểu cách dùng thật. Clio phân cụm hội thoại thực tế để biết người dùng làm gì, với chi phí thấp và có bảo vệ quyền riêng tư [10].

3 Dữ liệu

3.1 Nguồn và cách thu thập

Kiểm tra ngày 05/11/2026 trên một máy Ubuntu có Codex và Antigravity. Máy này không có Claude Code CLI (không có `~/.claude`), nên định dạng Claude Code được kiểm tra trên một phiên Claude Agent SDK chạy trên cloud. Chỉ đọc tên trường và con số, không đọc nội dung hội thoại, lệnh hay mã nguồn.

CÔNG CỤ	VỊ TRÍ LOG	ĐỊNH DẠNG	ĐÃ XEM
Codex	`~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL; bản cũ nén `.zst`	7 file
Claude Code	`~/.claude/projects/<dự án>/<phiên>.jsonl`	JSONL	1 phiên
Antigravity	`~/.gemini/antigravity/`; transcript trong từng workspace	SQLite, log văn bản	DB, log CLI, tài liệu hook

Bảng 1. Nguồn dữ liệu đã kiểm tra.

3.2 Trường dữ liệu theo công cụ

THÔNG TIN	CODEX	CLAUDE CODE	ANTIGRAVITY
Token theo từng lần gọi	Có	Có	Không thấy
Cache đọc / ghi	Có	Có, tách 5 phút / 1 giờ	Không thấy
Token suy luận	Có	Có	Không thấy
Model, mức suy luận	Có	Có	Qua hook
Thời gian	Có	Có	Một phần
Lỗi	exit_code, error	is_error	Qua hook
Phiên phụ, sub-agent	Có	Có	Có
Chi phí USD	Tự tính	Có, tổng phiên	Tự tính

Bảng 2. Thông tin có trong log. Codex: `token_usage_record.usage` (input, cached_input, cache_write_input, output, reasoning_output), `turn_context.model/effort`, `exit_code`, `task_complete.duration_ms/error`. Claude Code: `message.usage` (input, cache_read_input, cache_creation_input, output, output_tokens_details.thinking_tokens), `is_error`, `isSidechain`, dòng `cost-state` với `totalCostUSD`.

3.3 Hai lỗi dễ mắc khi đọc log

Đếm trùng ở Claude Code. Một phản hồi của model bị ghi thành nhiều dòng (mỗi khối thinking, text, tool là một dòng), và dòng nào cũng mang cùng số `usage`. Phiên kiểm tra có 461 dòng nhưng chỉ 254 phản hồi thật. Phải gộp theo `message.id`, nếu không chi phí bị cộng gần gấp đôi.

"Input" mang nghĩa khác nhau. Ở OpenAI (Codex), phần cache nằm trong `input_tokens`. Ở Anthropic (Claude), `input_tokens` không gồm cache [12]. Phải chuẩn hóa theo nhà cung cấp trước khi áp công thức.

3.4 Quyền riêng tư

Log chứa toàn bộ hội thoại và mã nguồn. µToken chỉ giữ con số và trạng thái; nội dung bị bỏ ngay khi đọc và không rời khỏi máy. Riêng việc nhận diện lệnh test cần xem tên lệnh: làm trên máy, chỉ giữ kết quả đúng/sai.

4 Phương pháp đo

4.1 Quy trình 8 bước



Hình 1. Quy trình xử lý. Bước 1–6 chạy bằng script trên máy người dùng; AI chỉ tham gia ở bước 7.

Log có thể lớn hơn 500 MB, nên bước 2 đọc từng dòng, giải nén `.zst` khi đọc, và nhớ vị trí đã đọc để lần sau chỉ đọc phần mới. Bước 3 gộp mỗi lần gọi model thành một dòng chung cho mọi công cụ, gộp trùng và chuẩn hóa cache như mục 3.3.

4.2 Các chỉ số

CHỈ SỐ	CÔNG THỨC	TRẢ LỜI CÂU HỎI
Chi phí (USD)	công thức mục 2.2	Thực sự đã trả bao nhiêu?
Tỷ lệ cache	cache đọc ÷ tổng input	Phần lặp lại có bị trả đủ giá?
Tỷ lệ suy luận	suy luận ÷ output	Có nghĩ quá nhiều cho việc dễ?
Độ phình context	input lần gọi cuối ÷ lần gọi đầu	Đã đến lúc mở phiên mới?
Tỷ lệ lỗi	lệnh hoặc tool lỗi ÷ tổng	Mất bao nhiêu cho thử lại?
Chi phí ngầm	chi phí phiên phụ ÷ tổng	Phần người dùng không thấy?
Tỷ lệ thành công (SR)	task thành công ÷ task đã xác định	Việc có xong không?
Chi phí mỗi task thành công (CPST)	tổng chi phí ÷ số task thành công	Chỉ số so sánh chính, đọc cùng SR

Bảng 3. Tám chỉ số. Tổng input cho tỷ lệ cache: OpenAI = `input_tokens`; Anthropic = input + cache đọc + cache ghi.

4.3 Xác định task thành công

Log không có nhãn thành công, nên µToken suy ra từ tín hiệu có sẵn và ghi rõ mức chắc chắn. SR chính chỉ dùng hai trạng thái có bằng chứng; SR phụ thêm hai trạng thái "có thể", để kiểm tra kết luận có đổi khi nới tiêu chí không.

TRẠNG THÁI	ĐIỀU KIỆN
Thành công, có bằng chứng	Lệnh test/build cuối cùng sau lần sửa file cuối thành công; không có turn lỗi.
Thất bại, có bằng chứng	Turn kết thúc bằng lỗi hoặc bị hủy; lệnh test/build cuối thất bại.
Có thể thành công	Có sửa file, không lỗi, người dùng không hỏi lại ngay.
Có thể thất bại	Nhiều lệnh lỗi liên tiếp, hoặc người dùng hỏi lại trong 2 phút.
Chưa xác định	Không có tín hiệu nào ở trên.

Bảng 4. Trạng thái kết quả của một task.

4.4 Phân nhóm loại việc

Task được gán vào nhóm (ví dụ: hỏi đáp, sửa lỗi nhỏ, tính năng mới, refactor lớn) bằng luật trên metadata: số file sửa, số dòng thay đổi, số lệnh, số turn, có chạy test hay không. Không cần đọc nội dung. Đây là phần chưa được kiểm chứng và cần thử trên log thật (giả định).

4.5 Mức bình thường và cờ cảnh báo

Mức bình thường là trung vị của chính người dùng, cùng model, cùng mức suy luận, cùng nhóm việc; chỉ dùng khi có từ 20 task trở lên (giả định). Các ngưỡng dưới đây là điểm khởi đầu, chưa hiệu chỉnh.

CỜ	NGƯỠNG ĐỀ XUẤT	GỢI Ý
Cache thấp	< 50% sau lần gọi thứ 3	Không chèn nội dung thay đổi lên đầu prompt; giữ cố định danh sách tool.
Context phình	> ×5, hoặc > 70% cửa sổ context	Mở phiên mới hoặc tóm tắt.
Lỗi lặp	≥ 3 lệnh lỗi liên tiếp	Dừng lại, bổ sung thông tin môi trường.
Nghĩ quá nhiều	Tỷ lệ suy luận > 2× mức thường ở task ngắn	Hạ mức suy luận cho loại việc này.
Chi phí cao	> 3× mức thường	Xem các cờ khác để tìm nguyên nhân.
Chi phí ngầm lớn	Phiên phụ > 30% tổng	Xem lại tự duyệt hoặc sub-agent.

Bảng 5. Cờ cảnh báo và ngưỡng khởi đầu.

4.6 Vai trò của LLM judge

Judge nhận gói số liệu đã tính (chỉ số, cờ, trạng thái kết quả), không nhận nội dung, và trả về nguyên nhân chính, bằng chứng (trích chỉ số), lời khuyên và mức chắc chắn. Judge **không** tự tính số và **không** được hỏi "A hay B tốt hơn"; việc so sánh làm bằng thống kê ở mục 5.

Lý do: chỉ đổi thứ tự trình bày đã làm Vicuna-13B "thắng" ChatGPT ở 66/80 câu [8]; GPT-4 giữ nguyên kết luận khi đảo vị trí chỉ 65% số lần [7]; GPT-4 nhận ra văn của chính mình 73,5% số lần, và mức tự nhận ra đi cùng mức tự ưu ái [9].

Biện pháp: judge khác họ model với model được đánh giá; hỏi 3 lần lấy đa số khi không chắc, vì tăng hơn 3 lần không cải thiện thêm [8]; định kỳ cho người chấm lại một mẫu nhỏ; ghi phiên bản rubric vào mỗi kết luận.

5 Phương pháp so sánh

Để trả lời "cách dùng A có tốt hơn B không", µToken kiểm tra lần lượt 6 bước. Bước nào không qua thì dừng và kết luận "chưa đủ bằng chứng".

5.0 So trong cùng công cụ

"Opus trong Claude Code" và "GPT trong Codex" khác nhau cả ở model lẫn công cụ: cách gửi context, cách cache, cơ chế tự duyệt (như phiên `guardian_review` của Codex). Muốn biết riêng **model** nào tốt hơn thì phải so trong cùng một công cụ. Khi so khác công cụ, kết luận phải ghi là so **bộ "công cụ + model"**.

5.1 So cùng loại việc

Chỉ so trong cùng nhóm việc (mục 4.4), tính chênh lệch trong từng nhóm rồi mới gộp. Lý do: model mạnh thường được giao việc khó; gộp chung sẽ làm nó trông đắt và hay thất bại một cách oan. Tỷ lệ các loại việc cũng khác nhau nhiều giữa người dùng [10]. So theo cặp giảm sai số mà không tốn thêm dữ liệu [2].

5.2 Kiểm tra cỡ mẫu

Trước khi so, tính chênh lệch nhỏ nhất phát hiện được (MDE). Nếu chênh lệch quan tâm nhỏ hơn MDE thì không thể kết luận [2].

```
MDE ≈ (1,96 + 0,84) × sai số của chênh lệch = 2,8 × SE_chênh
số task cần cho mỗi bên ≈ 7,84 × 2 × SR × (1 − SR) ÷ MDE²
```

1,96 ứng với độ tin cậy 95%, 0,84 ứng với xác suất phát hiện 80%. Trong ví dụ của Miller, khoảng 200 câu chỉ phát hiện chắc chênh lệch từ khoảng 13 điểm trở lên [2].

5.3 Xét hai trục: thành công và chi phí

A tốt hơn B khi SR không thấp hơn và CPST không cao hơn, với ít nhất một trục tốt hơn rõ. Nếu A thành công nhiều hơn nhưng cũng đắt hơn thì đó là đánh đổi. Các cách dùng không bị cách nào khác vượt trên cả hai trục tạo thành biên Pareto; chỉ chọn trong biên này [1].

5.4 Kiểm tra chênh lệch có thật

```
SE(SR)     = √( SR × (1 − SR) ÷ n )
KTC 95%    = SR ± 1,96 × SE
SE_chênh   = √( SE_A² + SE_B² )   (không theo cặp)
```

Khoảng tin cậy của chênh lệch chứa 0 thì chưa kết luận được. Task trong cùng phiên không độc lập, nên tính sai số theo phiên; sai số theo cụm có thể lớn gấp 3 lần [2]. Với CPST (phân phối lệch), dùng khoảng tin cậy bootstrap theo phiên (lựa chọn của nhóm).

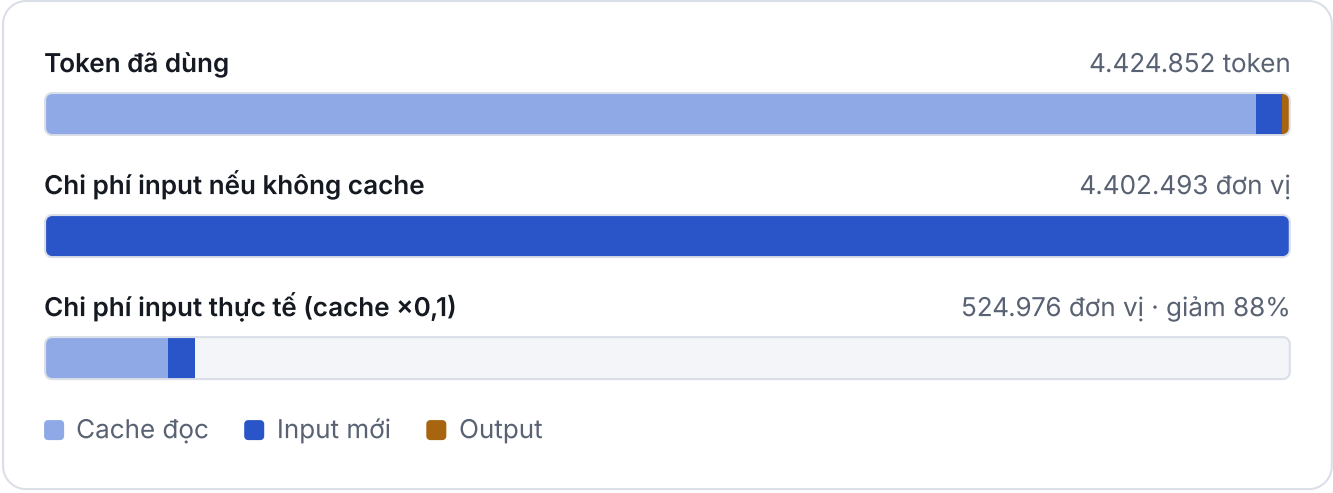
5.5 Kiểm tra độ ổn định

- **SR theo tuần:** dao động lớn nghĩa là khó đoán trước.
- **P90 ÷ P50 chi phí mỗi task:** đuôi dài nghĩa là thỉnh thoảng có task "cháy" tiền. Dùng trung vị vì chi phí lệch mạnh.
- **Tỷ lệ task phải làm lại:** đúng ngay hay phải thử nhiều lần.

Lý do: trên τ-bench, gpt-4o thành công khoảng 61% ở một lần thử, nhưng dưới 25% khi phải thành công cả 8 lần trên cùng task [3].

6 Kết quả trên log thật

6.1 Một phiên Codex



Hình 2. Phiên Codex, `gpt-5.6-sol`, effort `low`, 9 turn, 63 lần gọi. Một đơn vị = giá một token input thường.

CHỈ SỐ	GIÁ TRỊ	NHẬN XÉT
Tỷ lệ cache	97,9%	Rất tốt
Input trong tổng token	99,5%	Chi phí nằm chủ yếu ở phần đọc
Tỷ lệ suy luận	20,4%	Thấp, hợp với effort low
Độ phình context	×4,9	22,5k → 111k token mỗi lần gọi
Lệnh thất bại	6 / 35	Nguồn lãng phí đáng xem
Turn lỗi	0 / 9	Không có

Bảng 6. Chỉ số của phiên Codex.

6.2 Một phiên định dạng Claude Code

CHỈ SỐ	GIÁ TRỊ	NHẬN XÉT
Model	`claude-opus-5-5`	254 phản hồi, 277 lần gọi tool
Tỷ lệ cache	98,7%	Rất tốt; toàn bộ cache ghi loại 1 giờ
Tỷ lệ suy luận	13,5%	Thấp
Chi phí do log ghi	$42,99	Phiên nghiên cứu dài
Model phụ chạy nền (Haiku)	$0,018	Chi phí ngầm, nhỏ
Kết quả tool báo lỗi	9	Trên 277 lần gọi tool

Bảng 7. Chỉ số của một phiên Claude Agent SDK (định dạng Claude Code), chạy trên cloud. Bản CLI trên máy có thể thiếu vài trường.

6.3 Chi phí ngầm của Codex

Codex tạo phiên phụ để tự duyệt thao tác (`thread_source = guardian_review`, model `codex-auto-review`). Một phiên duyệt đo được khoảng 6,4 triệu token input, nhiều hơn phiên làm việc chính ở mục 6.1 (khoảng 4,4 triệu). Hai số đến từ hai phiên khác nhau nên chỉ minh họa, không phải tỷ lệ. Chi phí này thuộc về công cụ, không thuộc về model.

7 Model nào hiệu quả hơn?

Trả lời ngắn: chưa kết luận được với dữ liệu hiện có.

Đây là kết luận có chủ đích, không phải thiếu sót của phương pháp: µToken được thiết kế để nói "chưa đủ bằng chứng" thay vì đưa ra một thứ hạng sai.

7.1 Vì sao chưa kết luận được

1. **Mỗi model chỉ có một phiên.** Không tính được khoảng tin cậy, không đo được độ ổn định.
2. **Hai phiên làm hai việc khác nhau.** Vi phạm bước 5.1 (cùng loại việc).
3. **Hai phiên ở hai công cụ khác nhau.** Vi phạm bước 5.0: chênh lệch có thể do công cụ chứ không do model.
4. **Chưa có nhãn thành công.** Chưa tính được SR và CPST, nên chỉ so được các chỉ số quá trình (cache, suy luận), không so được hiệu quả.

7.2 Cần bao nhiêu dữ liệu

CHÊNH LỆCH SR MUỐN PHÁT HIỆN	TASK MỖI MODEL, MỖI NHÓM VIỆC	GHI CHÚ
20 điểm	~63	Chỉ thấy khác biệt rất lớn
15 điểm	~112	
10 điểm	~251	Mức hợp lý cho người dùng nặng
5 điểm	~1.004	Thường chỉ đạt khi gộp nhiều người dùng

Bảng 8. Cỡ mẫu cần thiết, giả sử SR ≈ 80%, độ tin cậy 95%, xác suất phát hiện 80%, so không theo cặp, theo công thức mục 5.2. Nếu task trong cùng phiên tương quan, cần nhiều hơn.

Hệ quả cho sản phẩm: với một người dùng, µToken thường chỉ kết luận được ở loại việc họ làm nhiều nhất. So sánh model ở mức tinh hơn cần gộp dữ liệu nhiều người dùng (có sự đồng ý), hoặc chấp nhận kết luận ở mức "rẻ hơn, chưa thấy kém hơn".

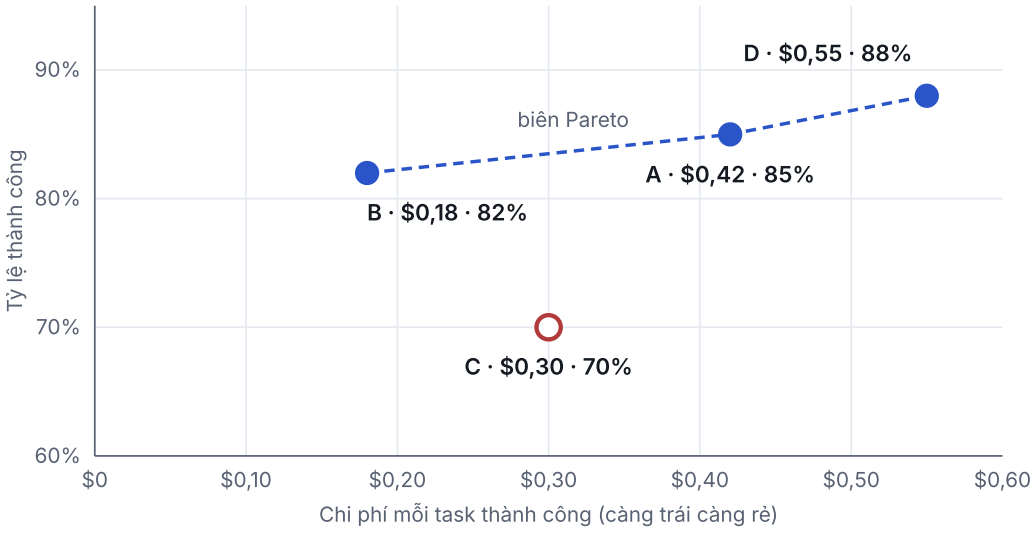
Điều **có thể** so ngay với dữ liệu ít: các chỉ số quá trình như tỷ lệ cache, độ phình context, tỷ lệ lỗi. Đây là chẩn đoán cách dùng, không phải xếp hạng model.

8 Ví dụ minh họa

Số liệu trong mục này là giả định

Dùng để minh họa cách đọc kết quả khi đã đủ dữ liệu. Nhóm việc: sửa lỗi nhỏ; cùng một công cụ; 60 task mỗi cách dùng.

8.1 Biên Pareto



Hình 3. Bốn cách dùng (giả định). A: model lớn, suy luận cao. B: model lớn, suy luận thấp. C: model nhỏ, suy luận cao. D: model lớn nhất. C bị B vượt trên cả hai trục nên nằm ngoài biên.

8.2 Khoảng tin cậy và độ ổn định

CÁCH DÙNG	SR ± KTC	CPST	SR THEO TUẦN	P90 ÷ P50
A · lớn, suy luận cao	85% ± 9	$0,42	87 · 80 · 87 · 87	2,1
B · lớn, suy luận thấp	82% ± 10	$0,18	87 · 73 · 87 · 80	3,8
C · nhỏ, suy luận cao	70% ± 12	$0,30	73 · 67 · 67 · 73	2,9
D · lớn nhất	88% ± 8	$0,55	87 · 87 · 93 · 87	1,8

Bảng 9. Kết quả giả định, 60 task mỗi cách dùng, 15 task mỗi tuần.

1. **C bị loại:** B rẻ hơn và thành công nhiều hơn.
2. **A so với B:** chênh SR 3 điểm; SE_chênh ≈ 6,8 điểm; KTC 95% từ −10 đến +16, chứa 0, nên chưa thấy A thành công nhiều hơn B. B rẻ hơn 57%.
3. **Chưa nói được "bằng nhau":** MDE ≈ 2,8 × 6,8 ≈ 19 điểm; chỉ loại trừ được việc B kém hơn A từ khoảng 19 điểm trở lên.
4. **Độ ổn định:** B dao động mạnh hơn (73–87%/tuần; P90 ÷ P50 = 3,8).

Kết luận mẫu

Với "sửa lỗi nhỏ": nên dùng B. Rẻ hơn 57%, chưa thấy kém hơn A. Mức chắc chắn: trung bình. B kém ổn định hơn, nên xem các task tốn nhất. Chỉ chọn D nếu 6 điểm thành công thêm đáng giá gấp 3 lần chi phí.

9 Kết quả hiển thị cho người dùng

9.1 Bảng so sánh theo loại việc

Kết quả chính µToken hiển thị là một bảng cho mỗi nhóm việc. Bảng dưới là trạng thái **thật** với dữ liệu hiện có; các ô trống sẽ có số khi đủ task.

CÔNG CỤ · MODEL · SUY LUẬN	TASK	SR	CPST	CACHE	ỔN ĐỊNH	KẾT LUẬN
Codex · gpt-5.6-sol · low	1 phiên	—	—	97,9%	—	Chưa đủ dữ liệu
Claude Code · claude-opus-5-5	1 phiên	—	—	98,7%	—	Chưa đủ dữ liệu

Bảng 10. Bảng kết quả với dữ liệu thật hiện có. Chưa phân nhóm việc vì chưa có luật phân nhóm (mục 4.4).

9.2 Luật ra kết luận

KẾT LUẬN	ĐIỀU KIỆN
Tốt hơn	Không kém rõ trên trục nào (KTC), tốt hơn rõ trên ít nhất một trục.
Rẻ hơn, chưa thấy kém	CPST thấp hơn rõ; KTC chênh lệch SR chứa 0. Kèm MDE để người dùng biết mức chắc chắn.
Đánh đổi	Tốt hơn rõ một trục, kém rõ trục còn lại. Hiển thị giá của mỗi điểm thành công thêm.
Kém hơn	Bị một cách dùng khác vượt trên cả hai trục (ngoài biên Pareto).
Chưa đủ dữ liệu	Dưới 20 task trong nhóm, hoặc MDE lớn hơn 20 điểm.

Bảng 11. Luật gán kết luận. Ngưỡng 20 task và 20 điểm là giả định ban đầu.

Mức chắc chắn hiển thị kèm mỗi kết luận: **cao** khi MDE ≤ 10 điểm, **trung bình** khi 10–20 điểm, **thấp** khi trên 20 điểm (giả định). Mỗi kết luận cũng kèm cờ ổn định nếu P90 ÷ P50 > 3 hoặc SR theo tuần dao động quá 10 điểm.

10 Thảo luận

10.1 Vì sao tính như vậy

LỰA CHỌN	BẰNG CHỨNG	NGUỒN
Tính bằng tiền, không bằng token	Cache đọc chỉ ×0,1 giá input; phiên Codex có 97% token là cache nhưng chi phí input chỉ còn 12%	[12], mục 6.1
Đo theo task thành công	92%+ câu đúng đã đúng ở vòng đầu; thêm token không thêm độ chính xác	[4]
Xét thành công và chi phí cùng lúc	Agent cùng độ chính xác chênh ~100× chi phí	[1]
Khoảng tin cậy, cỡ mẫu	~200 câu chỉ phát hiện chênh lệch ≥ 13 điểm; đọc điểm thô có thể ra kết luận ngược	[2]
Sai số theo phiên	Sai số theo cụm có thể lớn gấp 3	[2]
Đo độ ổn định	61% ở một lần thử, < 25% khi phải thành công 8 lần	[3]
Theo dõi độ phình context	20 → 50 tài liệu chỉ thêm ~1–1,5%	[6]
Judge chỉ nhận số, không so A/B	Đảo thứ tự lật 66/80 kết luận; GPT-4 nhất quán khi đảo vị trí 65%	[7][8]
Judge khác họ model	GPT-4 nhận ra văn của mình 73,5%	[9]
Gộp trùng theo message.id	461 dòng, 254 phản hồi thật	mục 3.3
So trong cùng công cụ	Công cụ có chi phí riêng (phiên tự duyệt 6,4M token)	mục 6.3
Mức thường của chính người dùng; ngưỡng cờ	Chưa có nghiên cứu trực tiếp	giả định

Bảng 12. Lựa chọn thiết kế và bằng chứng.

10.2 Các yếu tố có thể làm sai kết quả

- **Nhãn thành công suy ra:** có thể sai, đặc biệt ở việc không có test. Kiểm soát bằng SR chính/phụ (mục 4.3).
- **Phân nhóm việc sai:** làm hỏng bước 5.1. Cần kiểm tra luật trên dữ liệu thật.
- **Người dùng tự chọn model:** không phải thí nghiệm ngẫu nhiên; ngay cả trong cùng nhóm việc, task giao cho model mạnh có thể vẫn khó hơn.
- **Giá thay đổi:** CPST phụ thuộc bảng giá; khi giá đổi phải tính lại.
- **Định dạng log thay đổi:** theo phiên bản công cụ; parser cần kiểm tra phiên bản.
- **Nghiên cứu cũ:** phần lớn đo trên model 2021–2024; con số cụ thể có thể khác với model hiện nay.

10.3 Giới hạn

- Mới kiểm tra log Codex trên một máy và một phiên định dạng Claude Code.
- Antigravity không có số token cục bộ; transcript theo workspace chưa được kiểm tra.
- Chưa có dữ liệu nhiều phiên, nên mọi ngưỡng đều chưa hiệu chỉnh.

11 Triển khai

THÀNH PHẦN	CÔNG NGHỆ	NHIỆM VỤ
Desktop	Windows, Ubuntu; ví dụ Rust hoặc Go	Bước 1–6, dashboard đầy đủ; chạy nền, theo dõi thư mục log; đọc file lớn theo luồng; lưu SQLite cục bộ.
Server	Của nhóm	LLM judge và skill (bước 7); bảng giá model; đồng bộ số liệu tổng hợp; tùy chọn mức chung nhiều người dùng (có đồng ý).
Mobile	Flutter	Xem dashboard đã tính; nhận thông báo khi có cờ; không đọc log.

Bảng 13. Kiến trúc. Luồng dữ liệu: log (chỉ trên máy) → Desktop: số liệu → Server: judge, đồng bộ → Mobile: xem. Chỉ con số và cờ rời khỏi máy.

12 Kết luận và hướng tiếp

12.1 Kết luận

- **RQ1:** Log Codex và Claude Code đủ để tính chi phí, tỷ lệ cache, suy luận, độ phình context, lỗi và chi phí ngầm. Antigravity hiện chưa cung cấp số token cục bộ.
- **RQ2:** So sánh phải đi qua 6 bước: cùng công cụ, cùng loại việc, đủ mẫu, xét hai trục, khoảng tin cậy, độ ổn định. Với dữ liệu hiện có chưa thể nói model nào tốt hơn; cần khoảng 250 task mỗi model trong mỗi nhóm việc để phát hiện chênh lệch 10 điểm.
- **RQ3:** Mỗi lựa chọn đều có bằng chứng (Bảng 12), trừ mức bình thường và các ngưỡng, là giả định cần hiệu chỉnh.

12.2 Hướng tiếp

1. Kiểm tra Claude Code CLI trên máy thật và transcript Antigravity.
2. Viết parser bước 2–4 cho Codex và Claude Code; chạy trên toàn bộ phiên đang có.
3. Thử luật phân nhóm việc và nhận diện lệnh test trên log thật.
4. Thu dữ liệu từ vài người dùng thử để đặt mức bình thường và hiệu chỉnh ngưỡng.
5. Viết skill và rubric cho judge; chấm 30–50 ca và so với người chấm.
6. Thiết kế thí nghiệm có kiểm soát: cùng một bộ task chạy trên nhiều model trong cùng công cụ, để có so sánh model sạch.

Tài liệu tham khảo

1. Kapoor, S. và cộng sự (2024). AI Agents That Matter. arXiv:2407.01502.
2. Miller, E. (2024). Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations. arXiv:2411.00640.
3. Yao, S. và cộng sự (2024). τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045.
4. Chen, X. và cộng sự (2024). Do NOT Think That Much for 2+3=? On the Overthinking of o1-Like LLMs. arXiv:2412.21187.
5. Chen, L., Zaharia, M., Zou, J. (2023). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176.
6. Liu, N. F. và cộng sự (2023). Lost in the Middle: How Language Models Use Long Contexts. TACL.
7. Zheng, L. và cộng sự (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS Datasets and Benchmarks.
8. Wang, P. và cộng sự (2023). Large Language Models are not Fair Evaluators. arXiv:2305.17926.
9. Panickssery, A. và cộng sự (2024). LLM Evaluators Recognize and Favor Their Own Generations.
10. Tamkin, A. và cộng sự (2024). Clio: Privacy-Preserving Insights into Real-World AI Use. arXiv:2412.13678.
11. Liang, P. và cộng sự (2022). Holistic Evaluation of Language Models (HELM).
12. Anthropic; OpenAI. Tài liệu Prompt caching (truy cập 11/2026).

Số liệu log: kiểm tra ngày 05/11/2026. Mục 8 dùng số giả định. Nhãn (giả định) đánh dấu lựa chọn của nhóm chưa có nghiên cứu trực tiếp.